C

INF20125L

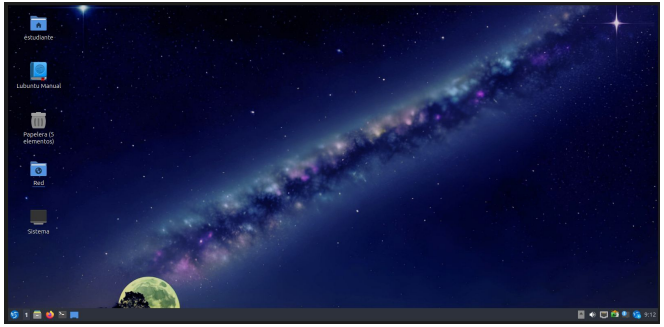
Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Volvamos a iniciar lubunto

Utilizaremos la distribución **lubuntu** en los laboratorios

Para ingresar en un laboratorio tienen que utilizar el usuario **.\test** y la contraseña **test**

Abriremos dentro de esta sesión la máquina virtual de oracle, Oracle VirtualBox y **iniciaremos la máquina** virtual seleccionada **lubuntu**



Lenguaje de programación C

El **lenguaje C** es un lenguaje de programación que **requiere compilación**, lo que lo hace un lenguaje de más **bajo nivel** que python y lo hace un lenguaje **más rápido** al momento de ejecución, aunque este tiene un **sintaxis** un poco más **compleja**, por ejemplo en **C** se tienen **tipos de dato estáticos**, una variable entera no se puede transformar a otro tipo, por lo que una variable entera siempre será entera. También en C tenemos acceso a los **punteros**, que apuntan a direcciones de memoria específica, además de la posibilidad que tiene de darnos de **gestión de memoria**.

En cambio **Python** es un lenguaje de **alto nivel** el cual es **interpretado** y de **sintaxis simple** y sus **variables** tienen **tipo dinámico**.

Diferencias entre python y C

Tenemos una estructura bien distinta entre ambos lenguaje, el ejemplo más simple solo **imprime “Hello World”** por pantalla, nos podemos percatar que **C** tiene un **include** y una **función** llamada **main**.

```c
#include <stdio.h>
int main() {
    printf("Hello World!\n");
    return 0;
}
```

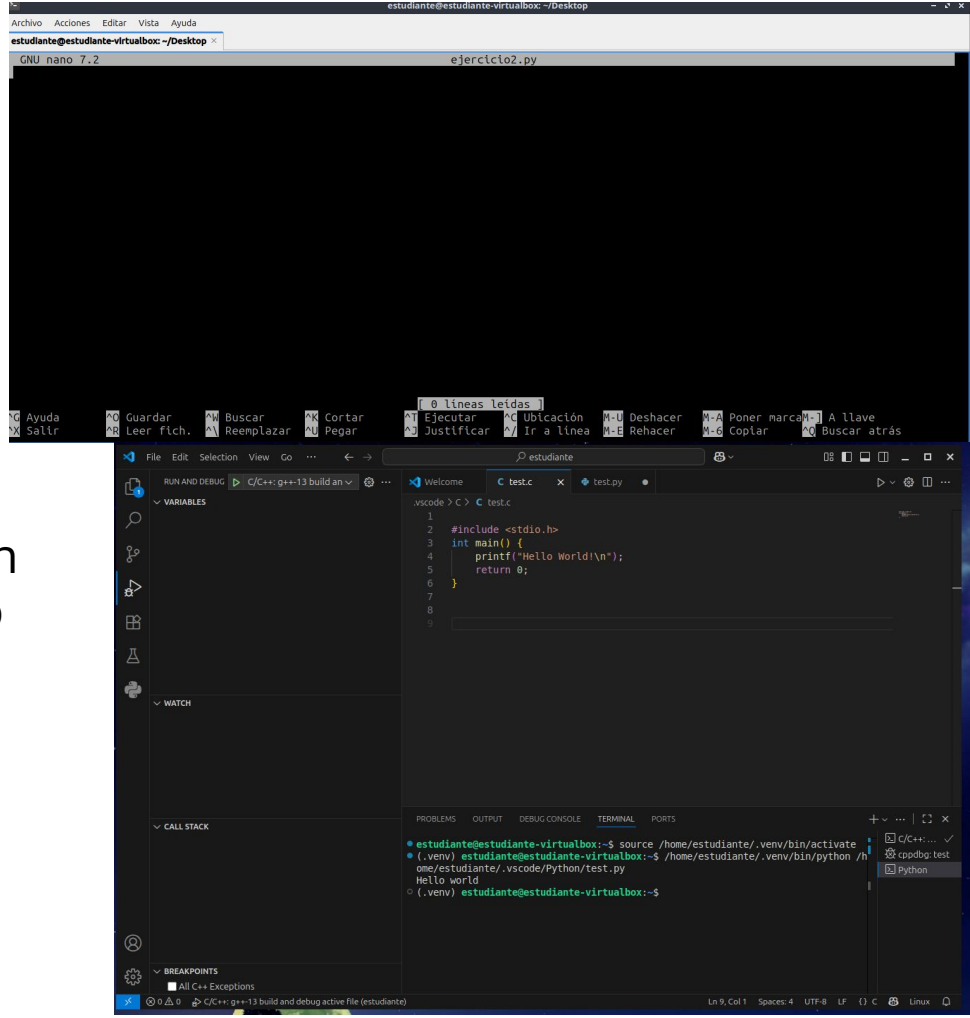
```python
print("Hello world")
```

Editores de texto

Nano

VSCode

Trabajaremos en nano en su mayoría, pero hay que aprender a utilizar otras herramientas de desarrollo que apoyen a nuestra programación. VSCode como otras herramientas tienen ayudas para el desarrollo de código, veamos un ejemplo.



Reglas de sintaxis en C

Cada **Sentencia termine** obligatoriamente con punto y coma (**;**)

Existen **bloques de código** iniciando con **{** y terminando con **}**

C distingue entre mayúsculas de minúsculas. **edad, Edad y EDAD son variables diferentes**.

Variables/funciones deben **comenzar** con una **letra** (a-z, A-Z) **o guión bajo** (_). No pueden empezar con números, ni contener espacios ni caracteres especiales (@, #, %).

No se pueden usar palabras reservadas del lenguaje (ej. int, if, while, return) como identificadores.

Se utilizan **//** para **Comentarios de una sola línea** y **/* ... */** para **bloques de comentarios**.

Todas las **variables** deben **declararse indicando su tipo** (int, float, char, double, etc.) antes de ser utilizadas

Nuestro primer programa en C

```c
#include <stdio.h>
int main() {
    printf("Hello World!\n");
    return 0;
}
```

Una vez que escriben este código, ¿ como se ejecuta ?
No es como python hay que compilarlo !

Compilación en C

La **compilación en C** transforma el **código fuente (.c)** legible por un humano en un archivo **ejecutable binario** (cómo .exe en Windows o binario sin extensión en Linux)

Este proceso tiene 4 etapas:

Preprocesamiento > interpreta y sustituye, #include y #define

Compilación > pasamos del código fuente preprocesado a lenguaje de ensamblador **(.s)**

Ensamblado > Convierte el código en ensamblador a código objeto **(.o)**

Enlazado (Linking) > Une los archivos objeto con las bibliotecas del sistema (como stdio.h), produce al archivo **ejecutable** final

‹stdio.h›

Es la biblioteca más fundamental de C. Define las funciones necesarias para que el programa se **comunique con el exterior** (pantalla y teclado).

Funciones:

printf() > Para mostrar datos en la pantalla.

scanf() > Para leer datos desde el teclado.

puts() / gets() > Para manejo básico de cadenas.

fopen() / fclose() > Para trabajar con archivos.

Diferencia entre ‹ › y " "

<archivo.h> Se usa para librerías estándar del sistema (las que ya vienen con el compilador). El compilador las busca en las carpetas del sistema.

"archivo.h" Se usa para librerías propias o locales que tú mismo hayas creado. El compilador las busca primero en la misma carpeta donde está tu código.

Flags (Banderas) comunes de compilación (GCC)

Generales y de Salida

-o [nombre] > Especifica el nombre del archivo ejecutable de salida.

-s > Se detiene tras la etapa de compilación, generando el archivo en código ensamblador **.s**

-c > Realiza la compilación y ensamblado, pero se detiene antes del enlazado, generando solo archivos objeto **.o**

Flags (Banderas) comunes de compilación (GCC)

Advertencias y depuración

-Wall > Activa casi todas las advertencias del compilador para detectar errores comunes de lógica o sintaxis.

-g > Incluye información de depuración en el ejecutable, necesaria para usar herramientas como GDB.

-Werror > Trata todas las advertencias como errores, deteniendo la compilación si encuentra alguna.

Flags (Banderas) comunes de compilación (GCC)

Optimización

-O0 > Sin optimización (predeterminado); ideal para depurar código.

-O2 > Optimización moderada; equilibra la velocidad de ejecución sin aumentar excesivamente el tamaño del binario.

-O3 > Máxima optimización de velocidad; puede aumentar el tamaño del archivo ejecutable.

-Os > Optimiza el código para reducir el tamaño del archivo final en lugar de la velocidad

Compilación y ejecución

Para compilar deberemos utilizar una llamada como la siguiente

gcc archivo.c -o ejecutable

Ya con el **ejecutable** podemos ver al ocupar **ll** que tiene los **permisos x**, lo que significa que podemos ejecutarlo de manera directa de la siguiente manera:

./ejecutable

Traduzcamos nuestro segundo código

```
edad = 25
precio = 19.9999
es_valido = True
inicial = 'C'

print("Edad {}".format(edad))
print("Precio {}".format(round(precio,2)))
print("¿Es Válido? {}".format(es_valido))
print("Inicial {}".format(inicial))
```

Traducción

```c
#include <stdio.h>
#include <stdbool.h>

int main(){
    int edad = 25;
    float precio = 19.99;
    bool es_valido = true;
    char inicial = 'C';

    printf("edad %d\n",edad);
    printf("Precio %.2f\n",precio);
    printf("Es valido %d\n",es_valido);
    printf("Inicial %c\n",inicial);
}
```

¿ Qué ocurre con la salida de datos?

Utilizamos la referencia del tipo que estamos utilizando tanto para mostrar datos “printf”.

```
printf("edad %d\n",edad);
printf("Precio %.2f\n",precio);
printf("Es valido %d\n",es_valido);
printf("Inicial %c\n",inicial);
```

Si quisiéramos pedir los datos tendremos que utilizar scanf(); especificando el tipo utilizado, como por ejemplo:

scanf("%d", &edad);

Ejercicio I

Pidan los datos con scanf(); para cada uno de los datos existente y muestren los resultados finales

```c
#include <stdio.h>
#include <stdbool.h>

int main(){
    int edad = 25;
    float precio = 19.99;
    bool es_valido = true;
    char inicial = 'C';

    printf("edad %d\n",edad);
    printf("Precio %.2f\n",precio);
    printf("Es valido %d\n",es_valido);
    printf("Inicial %c\n",inicial);
}
```

SizeOF()

A diferencia que en python, nuestras variables tendrán una asignación de espacio de memoria directa cuando nosotros asignamos su tipo de dato, y tendrá este **tamaño asociado a cada variable**.

Si utilizamos **sizeof(variable)** podremos observar el tamaño que se le ha asignado por memoria a estas. Un ejemplo claro e interesante de analizar es con las estructuras, el cual veremos por terminal.

Ejemplo:

```
int x = 10;

print("%ld\n",sizeof(x));
```

Structs

tipo de dato creado por el usuario que agrupa potencialmente distintos tipos de datos, esto nos permite manejar data compleja en una sola unidad.

Como por ejemplo:

struct Person {
  char name[50];
  int id;
  float salary;
};
…
struct Person p1
	typedef struct {
	  char name[50];
	  int id;
	  float salary;
	} Person;
	…
	Person p1;

Ejercicio 2

Cuando pesa la siguiente estructura:

```
struct Example {
    char a;   // 1 byte
    int b;    // 4 bytes
    char c;   // 1 byte
};
```

Revisen en C, cuánto es el valor que nos entrega al utilizar un sizeof()

C

INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Extras

Terminaste las asignaciones de ejercicios, puedes enfrentarte a los siguientes ejercicios y retos, entre ellos hay de investigación para lo que se nos viene, sé que algunos van más rápido así que pueden entretenerse en clases viendo otros problemas !

Ejercicio Extra

Crea un programa que pida la primera inicial del usuario, el precio de un producto y la cantidad de productos de ese tipo que compro, y luego muestre un resumen formateado. Usa printf para mostrar mensajes y scanf para leer datos. Utilice un struct para almacenar los datos del cliente.

Ejemplo de ejecución:

Introduce su inicial: **A**

Introduce el precio del producto: **500**

Introduce la cantidad de productos: **10**

Hola A, el total de su compra es de 5000

Investiguen el término Bash

Investigue el término bash y como se utiliza.

Cuando comience un archivo bash, recuerde colocar la siguiente línea al comienzo del archivo:

#!/bin/bash

Para poder ejecutar archivo bash tiene que darles permisos de ejecución al archivo con el comando

chmod

Para ejecutarlo

bash archivo.sh

Ejercicio Automatic testing

Cree un archivo bash para que ejecute su último ejercicio en C para que revise 2~3 archivos de entrada creados por usted, recuerde que puede jugar con las entradas para que vengan de un archivo utilizando la redirección de entrada

./ejercicio <input1 >output1

Ejercicio Automatic testing

Ahora que fue capaz de crear y probar las entradas y salidas de cada archivo, cree un programa que revise cada salida de archivo y verifique que es correcta, asuma que tiene un archivo de salida listo con los resultados que debería obtener.

La salida del bash debería informarnos cuántos resultados correctos (outputs) son idénticos a los obtenidos por su programa.

Aún aburrido en clases

Investigue que es git y github y suba todos sus avances a un repositorio, si llegó aquí, avísele al profe!, para tener ejercicios extra para la próxima semana !